

**TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A DISTANCIA**

Trabajo Práctico Integrador
Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Materia	Programación 1
Carrera	Tecnicatura Universitaria en Programación
Estudiantes	Nombre Apellido / Nombre Apellido
Docente	Nombre del docente
Fecha de entrega	DD/MM/AAAA

Tabla de contenido

1. Introducción	3
2. Objetivo del proyecto.....	3
3. Marco teórico.....	3
3.1 Listas.....	3
3.2 Diccionarios	4
3.3 Funciones	4
3.4 Condicionales	4
3.5 Estructuras repetitivas	5
3.6 Archivos CSV.....	5
3.7 Ordenamientos	5
3.8 Estadísticas básicas	5
4. Desarrollo del sistema.....	6
4.1 Modelo de datos	6
4.2 Funcionalidades implementadas	6
4.3 Validaciones y manejo de errores	7
4.4 Persistencia de datos	7
5. Decisiones técnicas y arquitectura.....	8
6. Diagrama de flujo	9
7. Capturas de pantalla	10
8. Dificultades encontradas	12
9. Conclusiones	12
10. Bibliografía / Webgrafía.....	12
11. Enlaces del proyecto	12

1. Introducción

El presente trabajo práctico integrador consiste en el desarrollo de una aplicación de consola en Python destinada a gestionar información sobre países. El sistema permite trabajar con un conjunto de datos cargado desde un archivo CSV, representando cada país mediante información básica como nombre, población, superficie y continente.

A partir de estos datos, el programa permite realizar operaciones de carga, búsqueda, filtrado, ordenamiento, actualización y generación de estadísticas. De esta manera, se integran distintos contenidos abordados durante la cursada de Programación 1, tales como listas, diccionarios, funciones, estructuras condicionales, estructuras repetitivas, manejo de archivos, validaciones y operaciones estadísticas básicas.

El proyecto fue desarrollado buscando que el código sea claro, modular y fácil de mantener. Para ello, se organizaron las funcionalidades en funciones específicas, procurando que cada una cumpla una responsabilidad concreta dentro del programa.

2. Objetivo del proyecto

El objetivo principal del proyecto es desarrollar una aplicación en Python que permita gestionar datos de países a partir de un archivo CSV, aplicando estructuras de datos y técnicas de programación vistas en la materia.

- Leer información desde un archivo CSV.
- Representar cada país mediante un diccionario.
- Almacenar el conjunto de países en una lista.
- Permitir agregar nuevos países.
- Permitir actualizar población y superficie de países existentes.
- Buscar países por nombre, admitiendo coincidencias parciales.
- Filtrar países por continente, rango de población y rango de superficie.
- Ordenar países por nombre, población y superficie.
- Calcular estadísticas básicas sobre el conjunto de datos.
- Implementar validaciones para evitar errores de ejecución.
- Organizar el código en funciones claras y reutilizables.

3. Marco teórico

3.1 Listas

Una lista en Python es una estructura de datos que permite almacenar varios elementos dentro de una misma variable. Las listas son ordenadas, modificables y permiten contener distintos tipos de datos. En este proyecto, se utiliza una lista para almacenar el conjunto completo de países cargados desde el archivo CSV.

Cada elemento de la lista representa un país. Esto permite recorrer todos los países mediante estructuras repetitivas, aplicar filtros, ordenar los datos y calcular estadísticas.

```
países = [pais1, pais2, pais3]]
```

3.2 Diccionarios

Un diccionario es una estructura de datos que almacena información mediante pares de clave y valor. Cada clave permite acceder a un dato específico. En este proyecto, cada país se representa mediante un diccionario con las claves nombre, poblacion, superficie y continente.

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}
```

Esta estructura resulta adecuada porque cada país posee atributos claramente definidos. El uso de claves permite acceder a cada dato de forma legible, por ejemplo, pais["nombre"] o pais["poblacion"].

3.3 Funciones

Una función es un bloque de código reutilizable que cumple una tarea específica. Las funciones permiten dividir un problema grande en problemas más pequeños, favoreciendo la modularidad, la legibilidad y el mantenimiento del programa.

En el proyecto se utilizaron funciones para separar responsabilidades. Por ejemplo, existen funciones para cargar países desde el CSV, mostrar menús, pedir datos al usuario, buscar países, filtrar resultados, ordenar datos y calcular estadísticas.

```
def cargar_paises(nombre_archivo):  
    # Lee el archivo CSV y devuelve una lista de países.  
  
def pedir_entero_positivo(mensaje):  
    # Solicita un número entero positivo y valida la entrada.  
  
def calcular_promedio(paises, campo):  
    # Calcula el promedio de un campo numérico.
```

3.4 Condicionales

Las estructuras condicionales permiten tomar decisiones dentro de un programa. En Python se utilizan principalmente las instrucciones if, elif y else.

En este proyecto, los condicionales se utilizan para validar opciones de menú, controlar errores de entrada, verificar si existen resultados de búsqueda, comprobar si un país ya está registrado y decidir si se debe guardar o no el archivo CSV.

```
if opcion == "1":  
    opcion_agregar_pais(paises)  
elif opcion == "2":  
    opcion_actualizar_pais(paises)  
else:  
    print(MENSAJE_OPCION_INVALIDA)
```

3.5 Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar un bloque de código varias veces. En Python se utilizan principalmente los bucles for y while.

En el proyecto se utiliza for para recorrer la lista de países y procesar cada elemento. También se utiliza while para mantener activo el menú principal hasta que el usuario decida salir y para repetir solicitudes de datos cuando el usuario ingresa valores inválidos.

```
for pais in paises:
    mostrar_pais(pais)

while opcion != "0":
    mostrar_menu()
```

3.6 Archivos CSV

Un archivo CSV es un archivo de texto que almacena datos separados por comas. Este formato es útil para representar información tabular de forma simple. En el proyecto, el archivo CSV contiene los países iniciales del sistema.

```
nombre,poblacion,superficie,continente
Argentina,45376763,2780400,América
Japón,125800000,377975,Asia
```

El programa utiliza el módulo csv de Python para leer y escribir datos. La lectura se realiza con csv.DictReader, que permite interpretar cada fila como un diccionario. La escritura se realiza con csv.DictWriter, permitiendo guardar nuevamente la lista de países en el archivo.

3.7 Ordenamientos

Ordenar datos consiste en organizarlos según un criterio determinado. En este proyecto, los países pueden ordenarse por nombre, población o superficie, tanto de forma ascendente como descendente.

Para realizar los ordenamientos se utiliza la función sorted() de Python, indicando mediante el parámetro key el campo por el cual se desea ordenar.

```
sorted(paises, key=lambda pais: pais["poblacion"], reverse=True)
```

3.8 Estadísticas básicas

Las estadísticas básicas permiten obtener información resumida a partir de un conjunto de datos. En este proyecto, se calculan indicadores como el país con mayor población, el país con menor población, el promedio de población, el promedio de superficie y la cantidad de países por continente.

Para obtener valores máximos y mínimos se utilizan las funciones max() y min(). Para los promedios se acumulan los valores y se dividen por la cantidad de países. Para contar países por continente se utiliza un diccionario acumulador.

```
total = 0

for pais in paises:
    total += pais["poblacion"]

promedio = total / len(paises)
```

4. Desarrollo del sistema

4.1 Modelo de datos

El sistema representa cada país mediante un diccionario. Cada diccionario contiene cuatro campos principales:

- nombre: nombre del país.
- poblacion: cantidad de habitantes.
- superficie: superficie en kilómetros cuadrados.
- continente: continente al que pertenece.

El conjunto completo de países se almacena en una lista. Esta decisión permite recorrer, filtrar, ordenar y modificar los países de forma sencilla.

```
país = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}
```

4.2 Funcionalidades implementadas

El programa cuenta con un menú principal de consola que permite acceder a las siguientes funcionalidades:

- Agregar país.
- Actualizar país.
- Buscar país por nombre.
- Filtrar países.
- Ordenar países.
- Mostrar estadísticas.
- Mostrar todos los países.
- Guardar cambios y salir.
- Salir sin guardar.

La opción de filtrado incluye un submenú que permite filtrar por continente, rango de población y rango de superficie. La opción de ordenamiento incluye otro submenú que permite ordenar por nombre, población o superficie, eligiendo además si el orden será ascendente o descendente.

4.3 Validaciones y manejo de errores

El sistema incorpora distintas validaciones para evitar fallos durante la ejecución y ofrecer mensajes claros al usuario.

Durante la carga del CSV se validan:

- Archivo inexistente.
- Columnas faltantes.
- Campos vacíos.
- Valores numéricos inválidos en población o superficie.
- Registros incompletos.

Durante la interacción con el usuario se validan:

- Campos vacíos.
- Números inválidos.
- Números menores o iguales a cero.
- Países duplicados al agregar.
- Opciones inválidas en menús y submenús.
- Búsquedas sin resultados.
- Filtros sin resultados.

Además, cuando el CSV contiene registros inválidos, el programa informa al usuario que esos países no fueron cargados. Si luego el usuario elige guardar, el sistema solicita confirmación porque el archivo será sobrescrito únicamente con los países válidos.

4.4 Persistencia de datos

Al iniciar el programa, los datos se cargan desde el archivo `países.csv`. Durante la ejecución, los cambios realizados se mantienen en memoria dentro de la lista de países.

- Guardar cambios y salir: escribe la lista actual de países en el archivo CSV.
- Salir sin guardar: finaliza el programa descartando los cambios realizados durante la ejecución.

Esta decisión permite que el usuario controle si desea conservar o no las modificaciones realizadas.

5. Decisiones técnicas y arquitectura

El programa fue desarrollado en un único archivo `main.py`, organizado por secciones para facilitar la lectura. Se decidió mantener el proyecto en un solo archivo porque corresponde a un trabajo de Programación 1 y permite concentrar la lógica principal sin agregar complejidad innecesaria.

El código se encuentra dividido en funciones según su responsabilidad:

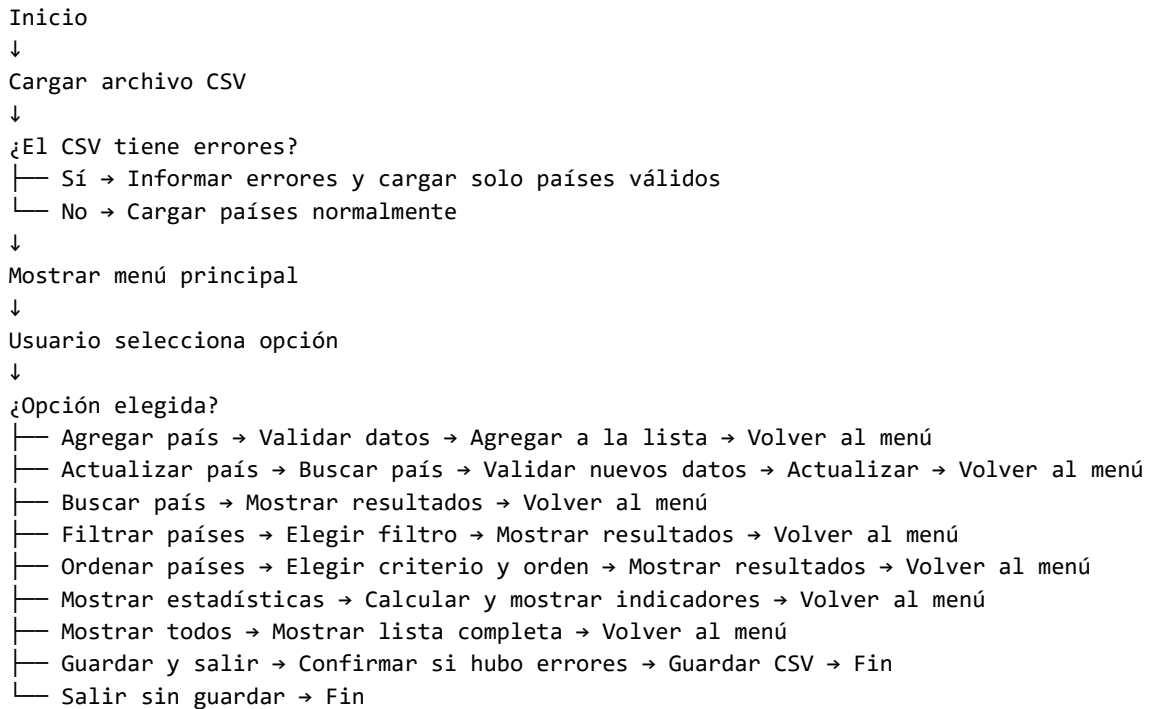
- Funciones de menú.
- Funciones auxiliares.
- Funciones de validación.
- Funciones de búsqueda.
- Funciones de filtrado.
- Funciones de ordenamiento.
- Funciones de estadísticas.
- Funciones de carga y guardado CSV.
- Funciones asociadas a opciones del menú.

La estructura principal del programa es la siguiente:

- Se carga el archivo CSV.
- Se valida el contenido.
- Se almacena la información válida en una lista de diccionarios.
- Se muestra el menú principal.
- El usuario selecciona una opción.
- Se ejecuta la función correspondiente.
- El programa vuelve al menú hasta que el usuario decide salir.
- Si el usuario elige guardar, se sobrescribe el CSV con los datos válidos actuales.

6. Diagrama de flujo

El flujo general del sistema puede representarse de la siguiente manera:



7. Capturas de pantalla

En esta sección se incorporan capturas de pantalla de la ejecución del programa. Reemplazar los recuadros por capturas reales antes de la entrega final.

Captura 1 - Menú principal

Insertar captura de pantalla correspondiente.

Captura 2 - Búsqueda de país por nombre

Insertar captura de pantalla correspondiente.

Captura 3 - Agregar país

Insertar captura de pantalla correspondiente.

Captura 4 - Actualizar país

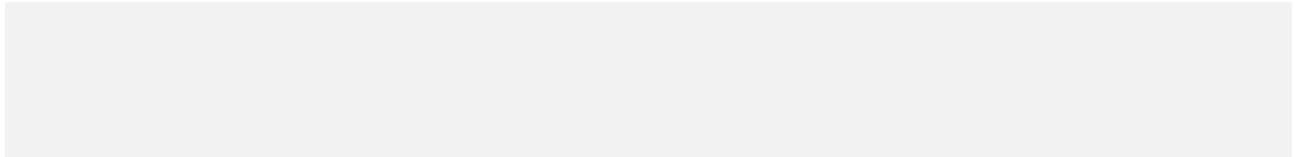
Insertar captura de pantalla correspondiente.

Captura 5 - Filtro por continente

Insertar captura de pantalla correspondiente.

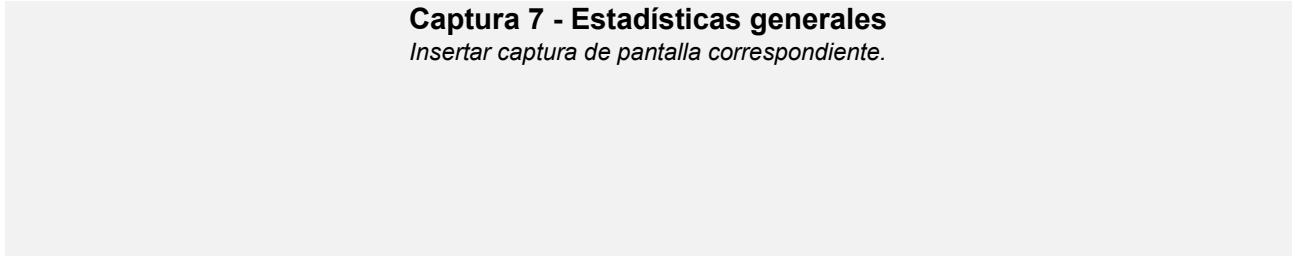
Captura 6 - Ordenamiento

Insertar captura de pantalla correspondiente.



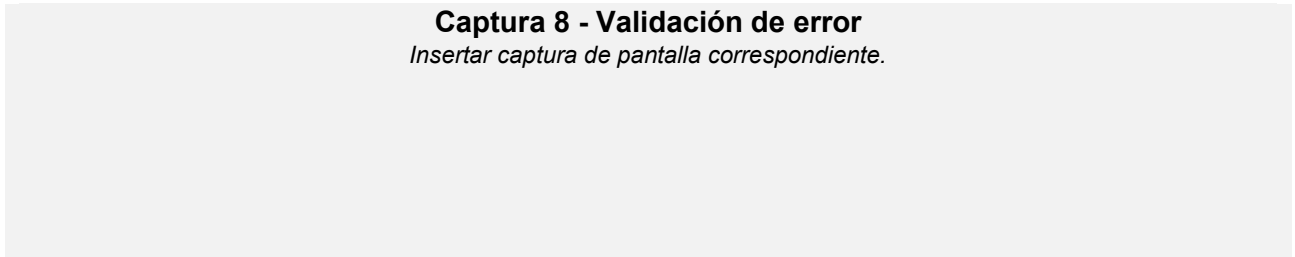
Captura 7 - Estadísticas generales

Insertar captura de pantalla correspondiente.



Captura 8 - Validación de error

Insertar captura de pantalla correspondiente.



8. Dificultades encontradas

Durante el desarrollo del proyecto surgieron distintas dificultades técnicas. Una de las principales fue el manejo correcto del archivo CSV, ya que los datos leídos desde el archivo ingresan como texto y fue necesario convertir población y superficie a números enteros para poder realizar cálculos, filtros y ordenamientos.

Otra dificultad fue el diseño de validaciones para evitar que el programa finalizara inesperadamente ante entradas incorrectas. Para resolverlo, se implementaron funciones específicas para pedir textos no vacíos y números enteros positivos, reutilizando esa lógica en distintas partes del sistema.

También se trabajó sobre la actualización de países mediante búsqueda parcial. Como una búsqueda podía devolver varios resultados, se agregó una función de selección para que el usuario pueda elegir el país correcto o cancelar la operación.

Finalmente, se resolvió el manejo de registros inválidos en el CSV. El programa carga únicamente los países válidos e informa los errores encontrados. Además, si el usuario intenta guardar luego de cargar un CSV con errores, se solicita confirmación para evitar sobrescribir el archivo sin advertencia.

9. Conclusiones

El desarrollo de este trabajo permitió integrar distintos contenidos vistos en Programación 1 dentro de una aplicación funcional. El uso de listas y diccionarios permitió representar de forma clara el conjunto de países y sus atributos. Las funciones facilitaron la organización del código, permitiendo dividir el problema en partes más pequeñas y reutilizables.

También se reforzó el uso de condicionales y estructuras repetitivas para construir menús, validar entradas y recorrer los datos. El manejo de archivos CSV permitió trabajar con persistencia de información, cargando datos desde un archivo externo y guardando modificaciones realizadas por el usuario.

Las funcionalidades de filtrado, ordenamiento y estadísticas permitieron aplicar operaciones útiles sobre el dataset. Además, el manejo de errores y validaciones ayudó a mejorar la robustez del programa y la experiencia del usuario.

Como conclusión general, el proyecto permitió afianzar la importancia de escribir código modular, legible y validado, así como también comprender cómo las estructuras de datos pueden utilizarse para resolver problemas concretos.

10. Bibliografía / Webgrafía

- Python Software Foundation. Documentación oficial de Python: <https://docs.python.org/es/3/>
- Python Software Foundation. Módulo csv: <https://docs.python.org/es/3/library/csv.html>
- Documentación y apuntes brindados por la catedra

11. Enlaces del proyecto

Repositorio GitHub: URL_DEL_REPOSITORIO

Video demostrativo: URL_DEL_VIDEO

Informe PDF: informe.pdf